

Sentinels

An AI-assisted passive security auditor for live websites

Project brief · URL scanning feature · Hackathon submission

Sentinels takes a single web address and returns a graded security report in about a second and a half. Five independent agents inspect the site concurrently — response headers, TLS certificate, DNS records, publicly reachable paths, and technology fingerprints — and their findings are turned into a deterministic 0–100 score, an A–F grade, a plain-English summary, and a deployment-readiness checklist. Every check is **passive**: it reads what a site already tells the public internet, and never sends attack traffic of any kind.

1. The problem

Shipping a website is easier than it has ever been, and the people shipping them are increasingly not security engineers — they are students, solo founders, and developers leaning heavily on AI-generated scaffolding. The result is a long tail of live sites with missing security headers, expired or misconfigured certificates, publicly readable `.env` files, exposed `.git` directories, and domains that anyone can spoof email from.

The tools that already exist sit at two unhelpful extremes:

- **Professional scanners** (Burp Suite, Nessus, OWASP ZAP) are powerful, but they are built for security professionals, they actively attack the target, and their output is a wall of jargon that assumes you already know what a finding means.
- **Single-purpose web checkers** (an SSL checker, a headers grader, a DNS lookup) each answer one narrow question. Getting a complete picture means visiting six sites, and none of them tell you what to actually do next.

There is a real gap in the middle: something a non-specialist can point at their own site, that is *safe to run against anything*, that covers the whole surface at once, and that explains its findings in language a non-specialist can act on.

2. What Sentinels does

The entire product is one interaction:

1. The user pastes a URL — `example.com` is enough; the input is normalised.
2. The URL is checked for reachability, so a typo fails fast with a clear message rather than returning a misleadingly clean report.
3. Five agents launch **concurrently** against the target.
4. Each agent's result streams back to the browser the moment it finishes, so the user watches the scan happen instead of staring at a spinner.
5. Findings are scored deterministically into 0–100 and A–F.
6. An LLM writes a plain-English executive summary on top of the finished, already-scored report.
7. A deployment-readiness checklist is derived from the findings.
8. The report is persisted at a permanent URL, and can be exported as PDF, JSON, or Markdown.

After the scan, two further AI features are available on demand: a per-finding **"Fix with AI"** explainer, and a **chatbot scoped to that one scan** that can only answer questions about the report in front of it.

3. The non-negotiable constraint: passive only

Scope rule, enforced throughout the codebase

Sentinels only ever performs actions an ordinary visitor's browser could perform: reading HTTP response headers, completing a TLS handshake, querying public DNS, and issuing plain `GET` requests to well-known public paths.

It never sends attack traffic. No SQL injection, no XSS payloads, no brute force, no fuzzing, no denial of service, no automated form submission, no authentication bypass attempts.

If a proposed feature requires sending something harmful, it is out of scope by definition.

This is a deliberate product decision, not a limitation of effort. It has three consequences that shape everything else:

- **It is safe to run against any target**, including sites the user does not own — no consent negotiation, no legal grey area, no risk of knocking something over.
- **It is fast.** A passive scan is a handful of network round-trips, not a crawl-and-probe campaign. The full scan finishes in ~1.3–1.5 seconds.
- **It is honest about its own limits.** Some important security properties simply cannot be observed passively, and the product says so explicitly rather than pretending (see the checklist tiers in §7).

4. Architecture

The system is a Python/FastAPI backend and a Next.js frontend, with SQLite for persistence. The backend is organised around a single idea: **an agent is a self-contained unit that takes a scan context and returns a list of findings**, and everything downstream speaks only in findings.

Layer	Responsibility
<code>agents/</code>	Five independent scanners. Each declares its own metadata and returns <code>Finding</code> objects. No agent knows any other exists.
<code>orchestrator.py</code>	Normalises the URL, checks reachability, runs all agents concurrently, assembles the report, persists it.
<code>scoring.py</code>	Pure functions: findings in, score / grade / severity counts out. No network, no clock, no randomness, no model.
<code>checklist/</code>	Maps findings onto a deployment-readiness checklist with explicit confidence tiers.
<code>ai/</code>	The LLM layer — summary, per-finding fix explainer, per-scan chatbot. Strictly additive.
<code>report/</code>	Export formats (PDF via headless Chromium, JSON, Markdown) behind a registry.
<code>storage/</code>	SQLite persistence for scans, findings, cached fixes, and chat history.

Concurrency is the core engineering claim

All five agents are dispatched with `asyncio.gather`, so a scan takes as long as the *slowest single agent*, not the sum of all five. This was measured directly rather than asserted: run sequentially, the five agents totalled 2597

ms, tracking the sum of their individual durations (2594 ms). Run concurrently, the same five totalled 1308 ms, tracking the single slowest agent (1297 ms) instead.

The streaming endpoint uses `asyncio.as_completed` rather than `gather`: every agent still starts at once, but each result is yielded to the client the instant that particular agent finishes, in real completion order. Adding a sixth agent is one line in the registry; nothing else in the orchestrator changes.

Agents can never crash a scan

Every agent runs inside a base-class wrapper that catches its own exceptions and reports them on `AgentResult.error`. A DNS timeout, a malformed certificate, or an unreachable path degrades that one agent's contribution — it never takes down the report. A partial report is always better than a 500.

5. The five agents

HEADERS

Security Headers

Checks whether the server sends the four most important security-related HTTP response headers. Follows redirects, because the header that matters is often only set on the final page rather than on an intermediate `http → https` hop.

- **Content-Security-Policy** — restricts what scripts, styles and frames the browser will load
- **Strict-Transport-Security** — prevents downgrade attacks from HTTPS to HTTP
- **X-Content-Type-Options** — stops browsers guessing a response's content type
- **X-Frame-Options** — blocks the page being embedded invisibly in another site (clickjacking)

TLS

TLS / Certificate

Opens a real TLS handshake using the standard library's `ssl` module and reads the certificate the server actually presents. The handshake is blocking, so it is moved off the event loop onto a worker thread.

- **HTTPS** — whether the site is served over TLS at all
- **Certificate validity** — expired, untrusted chain, or hostname mismatch
- **Certificate expiry** — warns when fewer than 30 days remain
- **Protocol version** — flags deprecated TLS 1.0/1.1 and SSLv2/v3

EXPOSURE

Sensitive File Exposure

Requests a small, fixed set of well-known paths that should never be publicly readable. Deliberately does *not* follow redirects — a genuinely exposed file is served directly, whereas a redirect to a login or 404 page means the file is not exposed. This is the agent where the passive boundary is tightest: it requests known public paths and reads the status code, and does not enumerate, fuzz, or download repository contents.

- `.env` — whether environment variables (API keys, database passwords) are readable
- `.git/HEAD` — whether the git repository is accessible, and therefore the entire source history downloadable

DNS

DNS / Email Security

Reads public DNS TXT records to determine whether the domain can be impersonated in email. A domain with no SPF and no DMARC can be spoofed by anyone — a phishing vector that has nothing to do with the website itself and is invisible to every header-based checker.

- **SPF record** — lists which servers are authorised to send email as this domain
- **DMARC record** — sets the policy when SPF fails (quarantine or reject)

RECON

Reconnaissance

Looks for information the site leaks about itself — the raw material an attacker uses to look up known vulnerabilities for a specific version.

- **Generator meta tag** — reveals the CMS or framework name and exact version
- **robots.txt** — flags **Disallow** entries containing admin, login, backup or config keywords, which advertise the very paths they intend to hide

6. The data model and the score

Every agent emits **Finding** objects with a common shape, which is what lets the orchestrator, scorer, AI layer and every export format share one vocabulary.

Field	Meaning
id	Stable slug, e.g. missing-hsts
title	Short human-readable title
category	Headers TLS DNS Exposure Recon
severity	Critical High Medium Low Info
status	fail (a real problem) warn (not ideal) pass (clean)
owasp	OWASP Top 10 mapping, e.g. A05:2021 - Security Misconfiguration
evidence	The raw technical detail actually observed
evidence_items	Structured, labelled proof — each tagged by kind (request, response headers, DNS record, certificate, HTML snippet)
description	Plain-language explanation of what is wrong
remediation	How to fix it

A crucial detail: agents emit a finding for every check they run, including the ones that **pass**. A clean site does not produce an empty report — it produces a report full of green. Passing checks cost zero points, but they are what makes the report evidence of work done rather than an absence of output.

Deterministic scoring

Start at 100. Subtract a fixed penalty for every non-passing finding.

Critical **-25** · High **-15** · Medium **-8** · Low **-3** · Info **0**

Clamped at 0. Grade from a fixed table: **A** ≥ 90, **B** ≥ 80, **C** ≥ 70, **D** ≥ 60, otherwise **F**.

No model is in the scoring loop, ever. The scoring module is pure functions that import nothing capable of non-determinism — no network, no clock, no randomness, no LLM. The same site produces the same score every time, and "why did this get a B?" always has a one-line answer: the score, and the table. This matters because a security score people cannot reproduce is a security score people will not trust.

7. Deployment readiness checklist

Alongside the score, findings are mapped onto a checklist answering a different question: *is this actually ready to ship?* The interesting design decision is that every row declares **how much Sentinels really knows**, in one of three tiers.

Tier	What it means	Examples
auto	Observed directly from a finding. Sentinels tested this and knows the answer.	HTTPS enforced · TLS certificate valid · certificate not expiring soon · HSTS set · CSP set · clickjacking protection set · <code>.env</code> not public · <code>.git</code> not public · SPF configured · DMARC configured
inferred	A weak passive signal. Explicitly labelled as not conclusive.	CMS/generator version not disclosed · <code>robots.txt</code> doesn't expose sensitive paths
self_attested	Cannot be observed passively at all. Sentinels never tests it — the developer answers, and the answer is recorded as theirs.	Input validation in place · rate limiting configured · authentication secured · dependencies audited for known CVEs

A readiness percentage is computed from the automatic items, producing a deployment status of **ready**, **caution**, or **blocked**. The self-attested tier is the honest part of the design: those four items are genuinely important and genuinely unobservable from outside, and a tool that quietly omitted them would be overstating what a passive scan can prove.

8. The AI layer

Three LLM-backed features sit on top of the scan. All three run through one shared client, and all three obey the same contract.

The AI only ever enriches — it never decides

If the API key is missing, or the call fails, or the provider rate-limits, the scan still produces a complete report: full score, full grade, every finding, the whole checklist. Only the enriched text is absent. The shared client returns `None` on *any* failure rather than raising, so a broken LLM can never propagate an exception into a scan. This was verified by removing the API key and re-running: the scan returned 200 with a complete report and an empty summary field.

1. Executive summary

Called after scoring is finished, and given the already-computed score, grade and findings. It writes a short plain-English summary that leads with the most serious problem if one exists. It cannot change the score — by the time it runs, the score is already fixed.

2. Per-finding "Fix with AI"

On demand for any single finding, returns a structured explanation: why the problem exists, its security impact, how it would be exploited *conceptually only* — *never a working exploit*, the recommended fix, general best practices, and framework-specific configuration examples (for example Apache and Nginx blocks). Results are cached in the database keyed by finding and prompt version, so the same explanation is not regenerated on every view.

3. Per-scan chatbot

A conversation scoped to one completed scan. It answers questions about that report and nothing else, with the full conversation persisted so the thread survives a page refresh.

9. API surface

Endpoint	Purpose
POST /scan	Run a full scan, return one complete report
GET /scan/stream	Same scan as Server-Sent Events — one event per agent as it finishes, then the final report
GET /agents	Metadata for every registered agent: name, purpose, list of checks, category
GET /scans	Paginated list of stored scans, newest first
GET /scans/{id}	Full stored report — hard refresh works, reports have permanent URLs
GET /scans/{id}/agents/{agent}	One agent's slice of a stored scan
GET /scans/{id}/checklist	Deployment readiness checklist
POST /scans/{id}/checklist/{item}	Record a developer's answer to a self-attested item
POST /scans/{id}/findings/{key}/fix	Generate (or return cached) AI fix guidance
POST /scans/{id}/chat	Ask one question about a completed scan
GET /scans/{id}/export/{format}	Export as pdf, json, or markdown
DELETE /scans/{id}	Delete a scan and its findings

10. Technology

Backend

- Python 3.13, FastAPI, uvicorn (ASGI)
- httpx — async HTTP client
- dnspython — DNS TXT lookups

Frontend

- Next.js (App Router), React 19, TypeScript
- Tailwind CSS v4
- Server-Sent Events for live scan progress

- `beautifulsoup4` — HTML parsing
- `stdlib` `ssl` + `sockets` — TLS handshake
- `Pydantic v2` — models and response validation
- `SQLite` — persistence
- `Playwright` / `headless Chromium` — PDF export
- `Groq API` — LLM inference
- `GSAP` + `Lenis` — landing-page motion
- `Framer Motion` — interaction design

Performance

- Full 5-agent scan: ~1.3–1.5 s
- Target was under 60 s — beaten by ~40×
- Concurrency gain measured, not assumed

11. Engineering principles

These are enforced conventions in the codebase, not aspirations:

Principle	Why
Passive checks only	Safe to run against anything; no consent or legal complexity; fast.
Scoring is deterministic	A score nobody can reproduce is a score nobody trusts. No model in the loop.
Agents never crash the scan	Each agent catches its own exceptions and reports them on its result. Partial reports beat 500s.
The AI layer is strictly additive	No API key must still mean a complete, useful report.
Agents are independent	No agent knows another exists. Adding one is a single registry line.
Passing checks are reported	A clean site should look audited, not unexamined.
Confidence is declared	Checklist tiers distinguish "we tested this" from "we're guessing" from "we cannot know".
Comments explain why, not what	The reader knows Python; they do not know the security domain.

12. What makes this different

- **Breadth in one request.** Headers, TLS, DNS, file exposure and fingerprinting are usually five separate tools. Here they are one URL and ~1.4 seconds.
- **Safe by construction.** Because nothing active is ever sent, the tool can be demonstrated live against any real site without preparation or risk — which also makes it usable by people who would never be allowed to run an active scanner.
- **Deterministic core, AI shell.** The parts that must be trustworthy (detection, scoring, grading) are ordinary testable code. The LLM is confined to explaining results it cannot influence. This is the opposite of the common pattern where a model is asked to decide something it cannot be held to.
- **Calibrated honesty.** The three-tier checklist openly separates what was measured from what was inferred from what was merely claimed by the developer. Most scoring tools blur exactly this line.
- **Evidence attached to every claim.** Each finding carries the raw material it was derived from — the actual header, the actual certificate, the actual DNS record — so nothing has to be taken on faith.
- **Built for the person who needs it.** Every finding carries a plain-language description and a remediation step, with deeper AI-generated guidance one click away.

13. Current status

The URL scanning feature described in this document is **complete and working end to end**: five agents, concurrent execution, streaming progress, deterministic scoring, the readiness checklist, all three AI features, persistence with permanent report URLs, and three export formats.

Work in progress beyond this document's scope: a second scanning mode that audits a public GitHub repository rather than a live site (secrets committed to history, dependency vulnerabilities, insecure configuration, repository hygiene), and continued design work on the landing experience.

Open areas for the URL scanner

The agent set is deliberately extensible — the orchestrator treats agents as an ordered list and adding one requires no changes outside the registry. Areas of the passive attack surface that the current five agents do not yet cover, and which remain open for expansion, include cookie attributes and session handling, CORS configuration, subresource integrity on third-party scripts, mixed-content loading, referrer and permissions policy, DNS records beyond email authentication, certificate transparency, HTTP method and redirect behaviour, and client-side dependency fingerprinting. Any addition must satisfy the same constraint as the existing five: it must be observable without sending a single hostile request.